

```

1  '''rrttriangles.py
2
3      This is the RRT skeleton code for the 2D triangular problem.
4
5      PLEASE FINISH WRITING THE CODE, especially where marked by FIXME.
6
7      Use RRT to find a path around polygonal obstacles.
8
9  '''
10
11  import matplotlib.pyplot as plt
12  import numpy as np
13  import random
14  import time
15
16  from math          import inf, pi, sin, cos, atan2, sqrt, ceil, dist
17
18  from shapely.geometry import Point, LineString, Polygon, MultiPolygon
19  from shapely.prepared import prep
20
21
22  #####
23  #
24  #   Parameters
25  #
26  #   Define the step size. Also set the maximum number of nodes.
27
28  DSTEP = 1.0
29
30  # Maximum number of steps (attempts) or nodes (successful steps).
31  SMAX = 50000
32  NMAX = 1500
33
34
35  #####
36  #
37  #   World Definitions (No Fixes Needed)
38  #
39  #   List of obstacles/objects as well as the start/goal.
40  #
41  (xmin, xmax) = (0, 10)
42  (ymin, ymax) = (0, 12)
43
44  # Collect all the triangles and prepare (for faster checking).

```

```

45 obstacles = prep(MultiPolygon([
46     Polygon([[7, 3], [3, 3], [3, 4], [7, 3]]),
47     Polygon([[5, 5], [7, 7], [4, 6], [5, 5]]),
48     Polygon([[9, 2], [8, 7], [6, 5], [9, 2]]),
49     Polygon([[1, 10], [7, 10], [4, 8], [1, 10]])))
50
51 # Define the start/goal states (x, y, theta)
52 (xstart, ystart) = (6, 1)
53 (xgoal, ygoal) = (5, 11)
54
55
56 #####
57 #
58 # Visualization Class (No Fixes Needed)
59 #
60 # This renders the world. In particular it provides the methods:
61 # show(text = '') Show the current figure
62 # drawNode(node, **kwargs) Draw a single node
63 # drawEdge(node1, node2, **kwargs) Draw an edge between nodes
64 # drawPath(path, **kwargs) Draw a path (list of nodes)
65 #
66 class Visualization:
67     def __init__(self):
68         # Clear the current, or create a new figure.
69         plt.clf()
70
71         # Create a new axes, enable the grid, and set axis limits.
72         plt.axes()
73         plt.grid(True)
74         plt.gca().axis('on')
75         plt.gca().set_xlim(xmin, xmax)
76         plt.gca().set_ylim(ymin, ymax)
77         plt.gca().set_aspect('equal')
78
79         # Show the triangles.
80         for poly in obstacles.context.geoms:
81             plt.plot(*poly.exterior.xy, 'k-', linewidth=2)
82
83         # Show immediately.
84         self.show()
85
86     def show(self, text = ''):
87         # Show the plot.
88         plt.pause(0.001)
89         # If text is specified, print and wait for confirmation.
90         if len(text)>0:

```

```

91         input(text + ' (hit return to continue)')
92
93     def drawNode(self, node, **kwargs):
94         plt.plot(node.x, node.y, **kwargs)
95
96     def drawEdge(self, head, tail, **kwargs):
97         plt.plot([head.x, tail.x], [head.y, tail.y], **kwargs)
98
99     def drawPath(self, path, **kwargs):
100         for i in range(len(path)-1):
101             self.drawEdge(path[i], path[i+1], **kwargs)
102
103
104     #####
105     #
106     #   Node Definition (No Fixes Needed)
107     #
108     class Node:
109         #####
110         # Initialization:
111         def __init__(self, x, y):
112             # Define/remember the state/coordinates (x,y).
113             self.x = x
114             self.y = y
115
116             # Define a parent (cleared for now).
117             self.parent = None
118
119         #####
120         # Planner functions:
121         # Compute the relative distance to another node.
122         def distance(self, other):
123             return sqrt((other.x - self.x)**2 + (other.y - self.y)**2)
124
125         # Check whether in free space.
126         def inFreespace(self):
127             if (self.x <= xmin or self.x >= xmax or
128                 self.y <= ymin or self.y >= ymax):
129                 return False
130             point = Point(self.x, self.y)
131             return obstacles.disjoint(point)
132
133         # Check the local planner - whether this connects to another node.
134         def connectsTo(self, other):
135             line = LineString([(self.x, self.y), (other.x, other.y)])
136             return obstacles.disjoint(line)

```

```

137
138 #####
139 # Utilities:
140 # In case we want to print the node.
141 def __repr__(self):
142     return ("<Point %5.2f,%5.2f>" % (self.x, self.y))
143
144 # Compute/create an intermediate node. This can be useful if you
145 # need to check the local planner by testing intermediate nodes.
146 def intermediate(self, other, alpha):
147     return Node(self.x + alpha * (other.x - self.x),
148               self.y + alpha * (other.y - self.y))
149
150
151 #####
152 #
153 # RRT Functions (to be fixed - see FIXME)
154 #
155 def rrt(startnode, goalnode, visual=None):
156     # Start the tree with the startnode (set no parent just in case).
157     startnode.parent = None
158     tree = [startnode]
159
160     # Function to attach a new node to an existing node: attach the
161     # parent, add to the tree, and show in the figure.
162     def addtotree(oldnode, newnode):
163         newnode.parent = oldnode
164         tree.append(newnode)
165         if visual:
166             visual.drawEdge(oldnode, newnode, color='g', linewidth=1)
167             visual.show()
168
169     # Loop - keep growing the tree.
170     steps = 0
171     while True:
172
173         # FIXME: Implement an RRT step!
174         # Break the while loop if the tree reaches the goal.
175
176         # Uniformly sample a target from entire space
177         # Add bias towards the goal
178         percent = 5
179         num = random.randint(1, 100)
180         if num <= percent:
181             target = goalnode
182         else:

```

```

183         target = Node(random.uniform(xmin, xmax),
184                        random.uniform(ymin, ymax))
185
186     last_dist = inf
187
188     # Loop through nodes in tree to find closest to target
189     for node in tree:
190         new_dist = node.distance(target)
191         if new_dist < last_dist:
192             last_dist = new_dist
193             nearest = node
194
195     if nearest.distance(target) == 0:
196         continue
197
198     # Identify the new node DSTEP away from nearest
199     # If nearest node is less than DSTEP away from target, new node is target
200     if DSTEP >= nearest.distance(target):
201         new_node = target
202     else:
203         dx = target.x - nearest.x
204         dy = target.y - nearest.y
205         d = nearest.distance(target)
206         new_node_x = nearest.x + DSTEP * (dx/d)
207         new_node_y = nearest.y + DSTEP * (dy/d)
208         new_node = Node(new_node_x, new_node_y)
209
210     # Check if nearest node connects to new node OR target and add to tree
211     if (new_node.inFreespace()) and (nearest.connectsTo(new_node)):
212         addtotree(nearest, new_node)
213         goal_dist = new_node.distance(goalnode)
214         if goal_dist <= DSTEP:
215             # If new node is less than DSTEP away from goal, check connection and add to tree
216             if new_node.connectsTo(goalnode):
217                 addtotree(new_node, goalnode)
218                 break
219
220     # Check whether we should abort - too many steps or nodes.
221     steps += 1
222     if (steps >= SMAX) or (len(tree) >= NMAX):
223         print("Aborted after %d steps and the tree having %d nodes" %
224               (steps, len(tree)))
225         return None
226
227     # Build the path.
228     path = [goalnode]

```

```

229     while path[0].parent is not None:
230         path.insert(0, path[0].parent)
231
232     # Report and return.
233     print("Finished after %d steps and the tree having %d nodes" %
234           (steps, len(tree)))
235     return path
236
237
238 # Compute the path cost
239 def pathCost(path):
240     cost = 0
241     for i in range(1, len(path)):
242         cost += path[i-1].distance(path[i])
243     return cost
244
245 # Post process the path
246 def postProcess(path):
247     shortpath = [path[0]]
248     for i in range(2, len(path)):
249         if not shortpath[-1].connectsTo(path[i]):
250             shortpath.append(path[i-1])
251     shortpath.append(path[-1])
252     return shortpath
253
254
255 #####
256 #
257 # Main Code (No Fixes Needed)
258 #
259 def main():
260     # Report the parameters.
261     print('Running with step size ', DSTEP, ' and up to ', NMAX, ' nodes.')
262
263     # Create the figure. Some computers seem to need an additional show()?
264     visual = Visualization()
265     visual.show()
266
267     # Create the start/goal nodes.
268     startnode = Node(xstart, ystart)
269     goalnnode = Node(xgoal, ygoal)
270
271     # Show the start/goal nodes.
272     visual.drawNode(startnode, color='orange', marker='o')
273     visual.drawNode(golnnode, color='purple', marker='o')
274     visual.show("Showing basic world")

```

```

275
276
277     # Run the RRT planner.
278     print("Running RRT...")
279     path = rrt(startnode, goalnode, visual)
280
281     # If unable to connect, just note before closing.
282     if not path:
283         visual.show("UNABLE TO FIND A PATH")
284         return
285
286     # Show the path.
287     cost = pathCost(path)
288     visual.drawPath(path, color='r', linewidth=2)
289     visual.show("Showing the raw path (cost/length %.1f)" % cost)
290
291
292     # Post process the path.
293     finalpath = postProcess(path)
294
295     # Show the post-processed path.
296     cost = pathCost(finalpath)
297     visual.drawPath(finalpath, color='b', linewidth=2)
298     visual.show("Showing the post-processed path (cost/length %.1f)" % cost)
299
300
301 if __name__ == "__main__":
302     main()

```